

Greedy or not

May 17, 2026

1 Introduction

There is a list of n numbers and two players who move alternately. On each move, a player removes either the first or last number from the list, and their score increases by that number and that number gets deleted. Both players try to maximize their scores and play optimally. A certain player wins if their score is strictly greater than the score of the other person.

2 Task

You have to write a code (in `Greedy_or_not.py` file) which takes input a number n which is the size of the list and then the element of the list. It then outputs which Player wins if both play optimally. We are not concerned about the time complexity of your solution.

2.1 Using the AutoGrader

Suppose the list is of size 9 and it contains the elements -5 5 9 -4 10 -9 0 3 -10. I have created a txt file `input1.txt` containing this input. The AutoGrader script automatically runs your Python solution against all the provided test cases and compares the output with the expected output. Place `AutoGrader.py`, `Greedy_or_not.py`, and the `testcases` folder (containing `input0.txt` through `input10.txt` and the corresponding `output0.txt` through `output10.txt`) in the same directory. Open the terminal in that directory and run the AutoGrader using the command

On Linux/Mac:

```
python3 AutoGrader.py
```

On Windows (PowerShell or Command Prompt):

```
python AutoGrader.py
```

This will run all 11 testcases and print PASSED or FAILED for each test case in the terminal itself. The AutoGrader uses `sys.executable` internally, so it will automatically use the same Python interpreter that you used to launch it, which means the same command works across all operating systems without modification.

2.2 Testcases

There are 11 testcases added with the expected output in the corresponding output.txt file. It is expected that testcases 0 to 7 should pass for your algorithm. Its fine if testcases 8,9 and 10 take a lot of time for your algorithm (i.e. your algorithm is exponential). These are only there for those who want to further optimize their algorithm (it should take less than a second with DP)

2.3 Additional Tasks (optional)

- (If you know Dynamic Programming) Optimize your code to solve it in quadratic time.
- Write your code in Python.